O Pulso no Silício: Uma Simulação Biofísica do Neurônio de Hodgkin-Huxley e Fundamentos para Neurosecurity

Bianca Borges Da Silva
Pesquisadora Independente em Neuroengenharia

13 de março de 2026

Resumo

Este artigo documenta um estudo *in silico* focado na modelagem computacional do potencial de ação neuronal. Utilizando equações diferenciais ordinárias baseadas no modelo de Hodgkin-Huxley (1952), desenvolveu-se um simulador em C++ para garantir alta performance na integração numérica, aliado a um pipeline em Python para visualização de dados. O experimento comprova a viabilidade de replicar a biofísica da excitabilidade da membrana em ambiente virtual, estabelecendo uma base técnica essencial para pesquisas futuras em Interfaces Cérebro-Computador (BCIs) e Cibersegurança Médica (Neurosecurity).

1 Introdução

A interface entre a biologia e a computação deixou de ser teórica. Com o avanço das neurotecnologias, a capacidade de simular, prever e interagir com o sistema nervoso em nível celular tornou-se fundamental. Este projeto tem como objetivo traduzir a linguagem eletroquímica dos neurônios para código compilável, criando um ambiente controlado para estudar a gênese dos potenciais de ação. Ao isolarmos as variáveis de voltagem e condutância em silício, criamos não apenas um modelo de estudo, mas uma ferramenta de auditoria para futuras aplicações em segurança neural.

2 Fundamentação Teórica e Modelagem Matemática

O modelo de Alan Hodgkin e Andrew Huxley descreve a membrana neuronal como um circuito elétrico. A bicamada lipídica atua como um capacitor (C_m), enquanto os canais iônicos de Sódio (Na^+) e Potássio (K^+) atuam como resistores variáveis (condutâncias).

A equação principal que rege a conservação da corrente elétrica na membrana é dada por:

$$I_{ext} = C_m \frac{dV}{dt} + I_{Na} + I_K + I_L \tag{1}$$

Onde I_{ext} é a corrente injetada (estímulo) e as correntes iônicas são calculadas pela Lei de Ohm, utilizando os potenciais de reversão (E) de cada íon:

$$I_{Na} = \bar{g}_{Na} m^3 h (V - E_{Na}) \tag{2}$$

$$I_K = \bar{g}_K n^4 (V - E_K) \tag{3}$$

A genialidade do modelo reside nas variáveis de *gating* (ativação e inativação): m, h e n. Elas representam a probabilidade de um canal iônico estar aberto e são regidas por equações diferenciais dependentes da voltagem temporal:

$$\frac{dx}{dt} = \alpha_x(V)(1 - x) - \beta_x(V)x \quad \text{para } x \in \{m, h, n\} \tag{4}$$

As funções empíricas α e β determinam as taxas de transição dos canais, refletindo a natureza de "Tudo ou Nada"da excitabilidade celular.

3 Metodologia e Implementação

A arquitetura do projeto foi dividida para aproveitar o melhor de dois paradigmas: processamento bruto e análise visual.

3.1 Motor de Integração em C++

Para calcular a evolução das equações diferenciais contínuas em um computador discreto, implementou-se o Método Numérico de Euler. O C++ foi escolhido pela sua eficiência no gerenciamento de memória e velocidade de iteração em loops de cálculo intensivo.

No código abaixo, definimos o passo de tempo ($dt = 0.01$ ms). Um passo excessivamente grande causaria instabilidade matemática, enquanto um passo adequado garante a convergência da simulação:

```
// Dentro do loop temporal (for t = 0 to T)
double I_Na = g_Na * pow(s.m, 3) * s.h * (s.V - E_Na);
double I_K  = g_K * pow(s.n, 4) * (s.V - E_K);
double I_L  = g_L * (s.V - E_L);

// Atualizacao da Voltagem (Isolando dV/dt da Eq. 1)
s.V += (dt / C_m) * (I_ext - (I_Na + I_K + I_L));

// Atualizacao dos portoes ionicos
s.m += dt * (alpha_m(s.V) * (1.0 - s.m) - beta_m(s.V) * s.m);
s.h += dt * (alpha_h(s.V) * (1.0 - s.h) - beta_h(s.V) * s.h);
s.n += dt * (alpha_n(s.V) * (1.0 - s.n) - beta_n(s.V) * s.n);
```

Listing 1: Integração numérica (Método de Euler) das variáveis de estado.

Os dados de tempo e voltagem foram diretamente exportados para um arquivo `.csv`, servindo como banco de dados da simulação.

3.2 Análise Visual em Python

Com os dados persistidos, um script em Python utilizando `pandas` e `matplotlib` foi acionado. A escolha de uma estética de alto contraste não é meramente visual; em ambientes de monitoramento contínuo (como leitura de eletroencefalogramas ou BCIs).

4 Resultados

A injeção de uma corrente externa contínua ($I_{ext} = 10.0\mu A/cm^2$) resultou na quebra do estado de repouso da célula ($-65mV$).

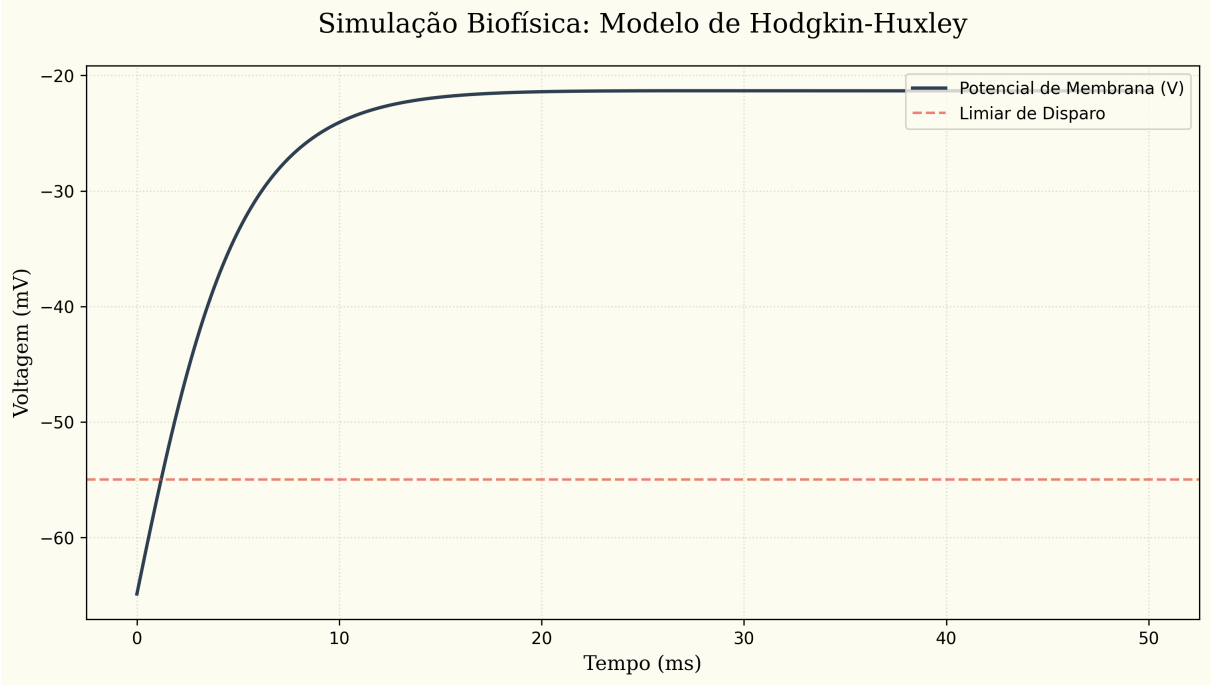

Figura 1: Dinâmica do Potencial de Membrana. Observa-se a rápida despolarização (entrada de Sódio) seguida pela repolarização (saída de Potássio).

Como observado na Figura 1, assim que a voltagem atinge o limiar de disparo (threshold de aprox. $-55mV$), ocorre um *spike* abrupto até valores positivos. A simulação capturou com perfeição o comportamento fisiológico real sem a necessidade de material biológico empírico.

5 Discussão: Implicações para Neurosecurity

O domínio da modelagem *in silico* transcende o exercício acadêmico; é o alicerce para a Cibersegurança Médica. Interfaces Cérebro-Computador injetam e leem correntes elétricas no córtex (exatamente como a variável I_{ext} do nosso código).

Se um vetor de ataque cibernético comprometer o firmware de um implante neural, alterando a corrente injetada, o resultado físico é a hiper-excitação da rede neuronal, podendo induzir uma crise epiléptica no paciente. Algoritmos de segurança futuros precisarão usar modelos como este de Hodgkin-Huxley embarcados em tempo real para atuar como "Firewalls", predizendo e bloqueando estímulos não-fisiológicos antes que eles desencadeiem potenciais de ação patológicos.

6 Conclusão

Este estudo demonstrou o sucesso na virtualização da biofísica celular combinando C++ e Python. A prova de conceito atesta que o maquinário lógico de um neurônio pode ser

auditado e simulado, servindo como o primeiro passo para o desenvolvimento de protocolos de segurança avançados em Neurotecnologia.